C++ refinery pricer: validation record

28 October 2026 | Illustrative refinery | Python remains the source of truth

Accept pricing parity; benchmark the stress kernel only

Both C++ pricing models reproduce every monthly Python collar. Maximum absolute difference across solved strikes and put/call premiums is 2.131628207280301e-14 USD/bbl. The comparison covers 48 product/month/model rows against a 1e-8 tolerance. No existing refinery calculation, source input, workbook or risk memo changed.

Check	Result
Pricing and Greeks pytest cases	57
Native CTest cases / assertions	1 / 8
Monte Carlo paths / seed	20,000 / 20261027
Measured repetitions	7
Python kernel median seconds	0.011707666010
C++ kernel median seconds	0.000603709000
Kernel speed-up	19.392896262765x
C++ CLI wall seconds, all seven repetitions	0.080195500006
Maximum MC summary difference	1.454286960766906e-15

Common shocks come from the original Python Random.gauss seed and correlation. Both kernels price the same annual average cracks, apply the first month's collars, sort and report the same fifth percentile and means. Kernel times exclude RNG, strike solving and I/O for both. CLI wall includes parsing and all seven repetitions. The original complete Python function took 0.243269125014s once; that broader workload is not the speed-up denominator.

Greeks and boundaries

Delta and gamma are forward derivatives. Vega is per full volatility unit; normal and Black-76 vegas have different units. Remaining-time derivative is the negative of calendar theta. Rho holds the forward fixed. Four put/call/model combinations pass finite-difference checks of all five Greeks. Native checks cover parity, non-negative gamma/vega, expiry and invalid inputs. Negative forwards are valid for normal pricing; Black-76 requires positive forward and strike.

Limits and reproducibility

Apple arm64, Apple clang 21.0.0, -O3, Python 3.12.14. Timing varies by machine, compiler and load and is not a production latency claim. The CLI uses Python-generated shocks rather than a native random generator. Expiry and zero-volatility kink Greeks use explicit API conventions. No calibration, executable quotes, early exercise or implied-vol solver is provided.

CMakeLists.txt builds the C++17 library and CLI. CI builds Release, runs CTest, then the Python parity tests. benchmark.json retains exact timings, the original model summary and Python-source SHA256 hashes; parity.csv retains all collar comparisons. See README.md for build commands and the function walkthrough.

The unchanged model recommends 80% cover. Its 30% crack-fall loss is 10%; Bachelier outage cost is 2.3013698630136973% and Black-76 outage cost is 1.5473073710677457% of budget margin.